

RECORD (STRUCT):

→ static data structure

ex:

Person

name: String
dob: String
addr: String
etc...

ARRAY

✓ → data structure (static)

✓ → holds fixed nr. of elements of the same type

✓ → elements occupy a contiguous memory block.

✓ → used as a basis for other data structures.

→ when creating an array, SPECIFY:

1) the TYPE of elements.

2) the MAX nr. of elements (CAPACITY)

⇒ memory occupied by array:

$$\text{CAPACITY} \times \text{sizeof}(\text{elem})$$

→ address of the first element.

ARRAY of boolean values

↳ addresses of the elements are displayed in big, b10.

ADVANTAGES OF ARRAYS:

- (H) elem. can be accessed in $\Theta(1)$ (constant time)

$$\text{address of } i^{\text{th}} \text{ elem} = \text{address of array} + i \times \text{sizeof}(\text{elem})$$

↳ indexing from 0

↳ indexing from 1: " - 1 - (i-1) - 1 - ...

DISADVANTAGES:

- we know from the start the number of elements.

~~ARE~~ DYNAMIC ARRAYS (vectors)

- their size can grow/shrink
- contiguous memory locations
- compute address of every element in $\Theta(1)$ time.

Dynamic Array:

cap: Integer
nrElem: Integer
elems: TElem[]

$\left\{ \begin{array}{l} \text{cap} \rightarrow \text{capacity (number of slots allocated for the array)} \\ \text{nrElem} \rightarrow \text{current capacity (actual nr. of elements stored)} \\ \text{elems} \rightarrow \text{actual array with cap slots for TElem allocated.} \end{array} \right.$

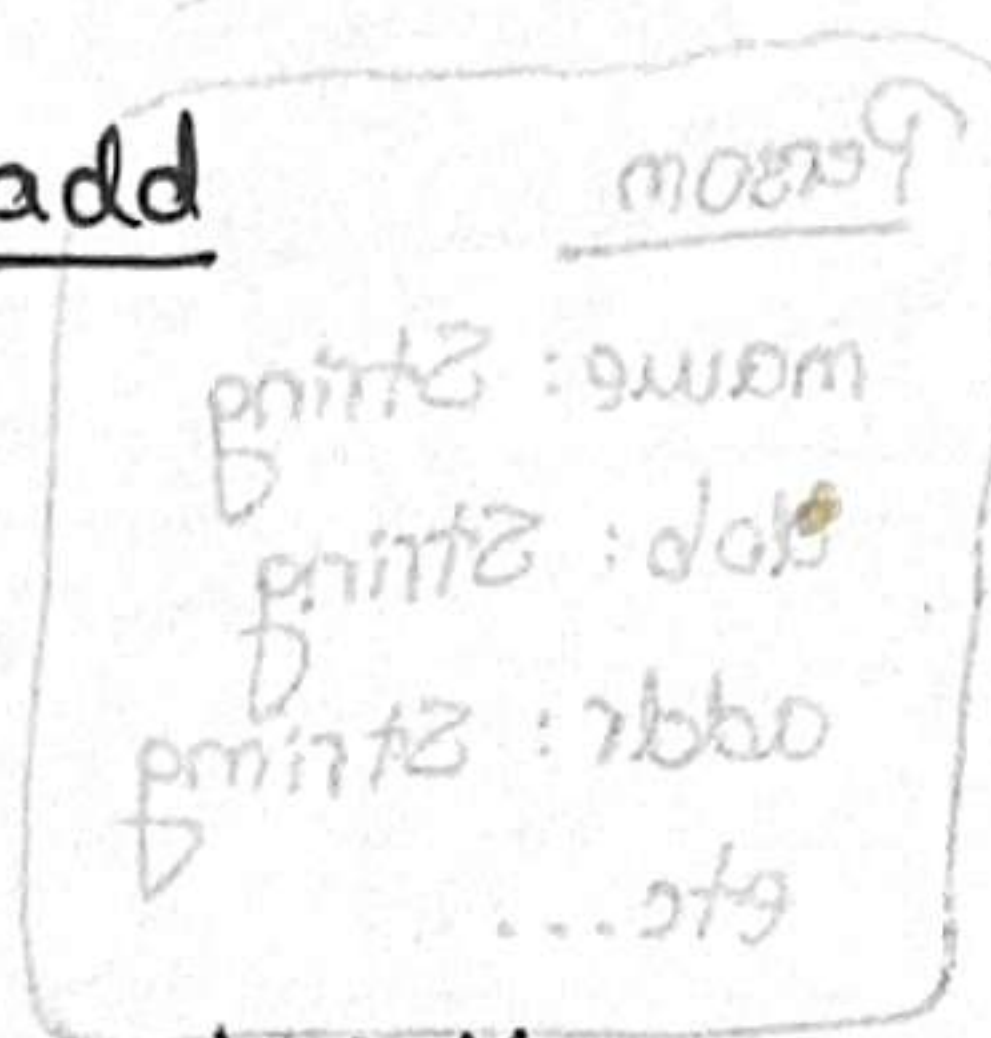
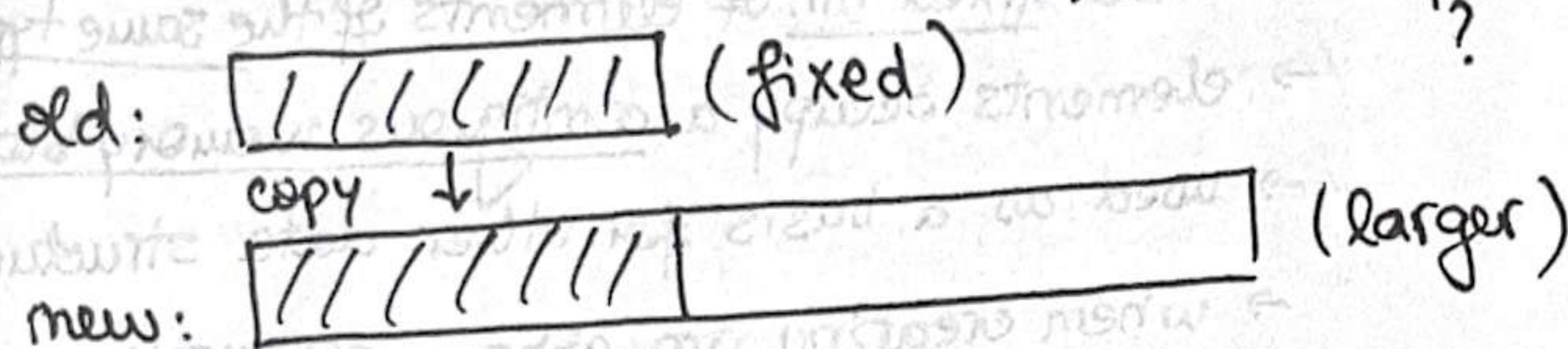
- have positions (no iterators needed)
- they are data structures (ADT) ~

- FULL ARRAY $\Rightarrow$ $nrElem = cap$.

- More elements needed? $\Rightarrow cap * 2 \Rightarrow$ resized.

RESIZE OPERATION:

$\rightarrow$ resize before add?



$\rightarrow$ resize after delete (be it's "too empty") $\Rightarrow$ shrink.

(to avoid occupying unused memory).

Why is Dynamic Array a DS?

- it describes how data is stored in computer
- it can be used as representation to implement $\neq$ ADT's.

Why not really an ADT?

- it has one single possible implementation $\rightarrow$??
- but we treat it as an ADT and discuss its interface.

Formal definition for Dynamic Array (ADT):

• Domain: $DA = \{da \mid da = (cap, nrElem, e_1, e_2, \dots, e_{nrElem}), cap, nrElem \in \mathbb{N}, nrElem \leq cap, e_i \text{ is of type } TElem\}$.

Interface:

Operations for DA:

1) • init(da, cp) $\leftarrow$ CONSTRUCTOR

$\rightarrow$ creates new, empty Dynamic Array with initial capacity (cp).

• pre: $cp \in \mathbb{N}^*$

• post: $da \in DA$

$da.cap = cp$

$da.nrElem = 0$

• throws: an exception if $cp = 0$ or $cp \leq 0$.

2) • destroy(da) $\leftarrow$ DESTRUCTOR

$\rightarrow$ destructor

• pre: $da \in DA$

• post: da was destroyed (memory occupied was freed).

3) size(da)

$\Theta(1)$

↳ nr. of elements of the DA.

• pre: $da \in DA$

• post: $size \leftarrow$ nr. of elements ...

4) getElement(da, i)

$\Theta(1)$

↳ return element from a position from the DA.

• pre: $da \in DA, 1 \leq i \leq da.mrElem$

• post: $getElement \leftarrow e, e \in TElem$

$e = da.e_i$ (element from position i)

• throws: exception if i not valid pos.

5) setElement(da, i, e)

$\Theta(1)$

↳ changes the element from a position to another value

• pre:

• post:

• throws: " — " —

look at the sequence of calls $\rightarrow$ compute average nr. of instructions ...

(not AC); in time, divided by sth; those costs that don't do resizing.

6) addToEnd(da, e)

$\Theta(1)$ amortized

↳ adds to the end of DA; if full, increases its capacity

• pre: $da \in DA, e \in TElem$

• post: $da' \in DA, da'.mrElem = da.mrElem + 1$

$da'.e_{da'.mrElem} = e$

$da.cap = da.mrElem$

↓ increase size

$da'.cap \leftarrow da.cap * 2$

ADD * PSEUDOCODE IMPLEMENTATION:

→ subalgorithm addToEnd(da, e):

if $da.cap = da.mrElem$ then

$da.cap \leftarrow da.cap * 2$

$newArray \leftarrow$ @ is a new array with $da.cap$ elements

for $i \leftarrow 1, da.mrElem$ execute

$newArray[i] \leftarrow da.elems[i]$

@ deallocate $da.elems$?

$da.elems \leftarrow newArray$

$\begin{cases} da.mrElem \leftarrow da.mrElem + 1 \\ da.elems[da.mrElem] \leftarrow e \end{cases}$

full capacity = $mr.elems$

no variable holds the address of the old array.

7) `addPosition(da, i, e)` $O(n)$

↳ adds elem to given position.

• pre: $da \in DA, \dots$

• post: $da' \in DA$

$da'.mrElem = da.mrElem + 1$

$da'.e_j = da.e_{j-1}, (\forall) j \dots$

• throws: exception if i is not a valid pos.

! $da.mrElem + 1$ is a valid position when adding new elem.

$\Rightarrow BC, AC, WC$

↓
resize

• valid position: $mr.$ from $1 \rightarrow mrElem$

• valid position when adding: mr from $1 \rightarrow mrElem + 1$.

• * 2 too much when resizing?

↳ we need to multiply always, at least with ≥ 1 . (in C++, Java)
↳ it is better to **RESIZE ONCE** (to capacity 100, for instance) than to go through a lot of $\neq$ capacities to get to 100.

what happens in memory:

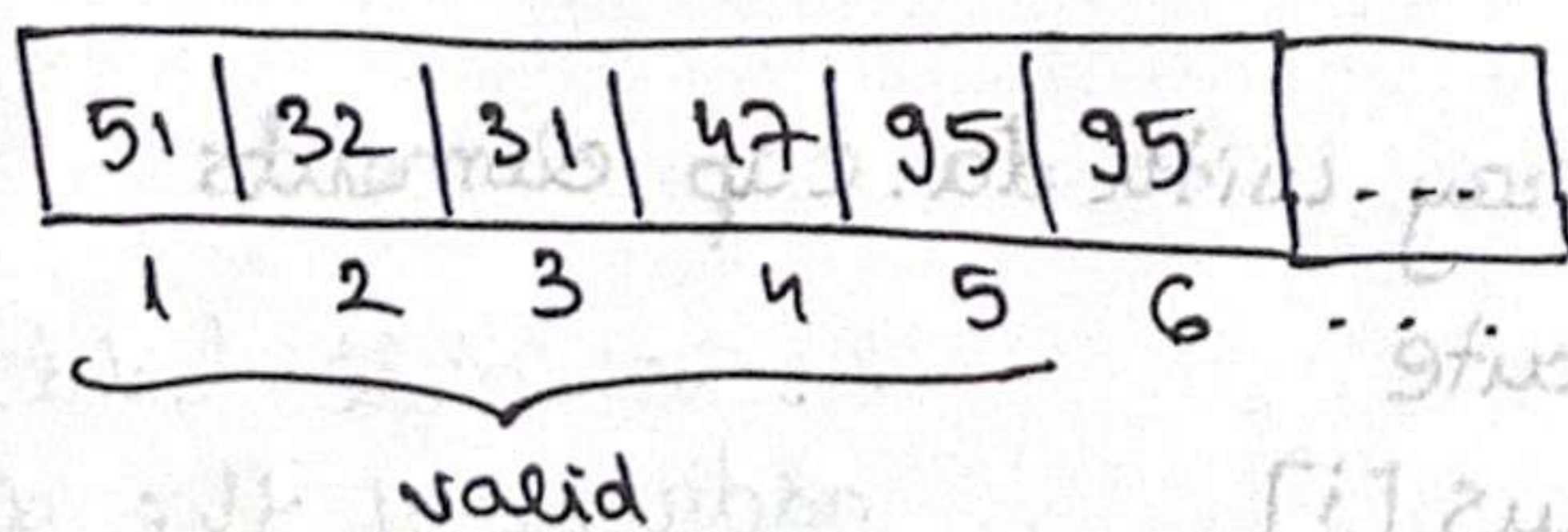
dynamic array $\rightarrow$ variable stores address of allocated memory.

$\rightarrow$ elements stored there $\rightarrow$ when capacity exceeded $\rightarrow$ new larger block is allocated, elements copied, old block freed $\rightarrow$ pointer updated to new address.

8) `deleteFromPosition(da, i)` $O(n)$

↳ ...

DELETE
←



Complexity: size: $\Theta(1)$
getElement: $\Theta(1)$
setElement: $\Theta(1)$
iterator: $\Theta(1)$
addToPosition: $O(m)$
deleteFromEnd: $\Theta(1)$
deleteFromPosition: $O(m)$
deleteGivenElement: $\Theta(m)$
addToEnd: $\Theta(1)$ amortized

→ better explanation:

BC for searching = WC for removing
(WC: searched elem. is on last position).

3) • iterator (da, it)

↳ returns an iterator for the DA.

• pre: $da \in DA$

• post: $(it) \in I$ is an iterator over da

↳ current elem from (it) = first element from da